

Learning-Guided Simulated Annealing for the Capacitated Vehicle Routing Problem

Anonymous submission

Abstract

Introduction

Background and Related Work

Problem Definition

The *Capacitated Vehicle Routing Problem (CVRP)* is defined on a complete undirected graph $G = (V, E)$, where $V = \{0\} \cup \mathcal{C}$ and $E = V \times V$ denote the sets of nodes and edges, respectively. Node 0 corresponds to the central depot, and $\mathcal{C} = \{1, \dots, N\}$ represents the set of N customers.

Each edge $(i, j) \in E$ is associated with a travel cost $c_{ij} = \|\mathbf{p}_i - \mathbf{p}_j\|_2$, the Euclidean distance between the 2D coordinates $\mathbf{p}_i, \mathbf{p}_j \in [0, 1]^2$, satisfying $c_{ij} = c_{ji}$. Each customer $i \in \mathcal{C}$ has a non-negative demand q_i , while the depot has zero demand ($q_0 = 0$). An unlimited fleet of identical vehicles, each with uniform capacity Q , is stationed at the depot; the number of routes m is therefore a decision variable unconstrained from above.

A route $R_k = (0, v_1, v_2, \dots, v_p, 0)$ is a simple cycle starting and ending at the depot, where $\{v_1, \dots, v_p\} \subseteq \mathcal{C}$ denotes the subset of customers served by vehicle k . A *solution* $s = \{R_1, \dots, R_m\}$ consists of a set of m routes. A solution s is *feasible* if and only if it satisfies the following conditions:

1. Each customer is visited exactly once by a single vehicle,

$$\sum_{k=1}^m \mathbf{1}_{\{i \in R_k\}} = 1, \quad \forall i \in \mathcal{C}.$$

2. For every route R_k , the total demand of its customers does not exceed the vehicle capacity Q , i.e.,

$$\sum_{i \in R_k \setminus \{0\}} q_i \leq Q, \quad \forall k = 1, \dots, m.$$

The set of *feasible solutions* is denoted by $\mathcal{F}$.

Let $x_{ij}(s)$ be a binary indicator variable with $x_{ij}(s) = 1$ if edge (i, j) is traversed in any route R_k of s , and $x_{ij}(s) = 0$

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

otherwise. The objective of the CVRP is to find a feasible solution $s^* \in \mathcal{F}$ that minimizes the total travel cost $C(s)$:

$$s^* = \arg \min_{s \in \mathcal{F}} \left\{ C(s) = \sum_{i \in V} \sum_{j \in V, i < j} c_{ij} x_{ij}(s) \right\}.$$

Proposed Method

Overview

We propose *Learning-Guided Simulated Annealing (LG-SA)*, a hybrid method that combines *Simulated Annealing (SA)* with a lightweight neural policy trained by *Reinforcement Learning (RL)*. Following the Neural Simulated Annealing framework (Correia, Worrall, and Bondesan 2023), the SA component orchestrates the global exploration of the solution space through its acceptance criterion and cooling schedule, while the neural network replaces the single blind component of SA — the uniform move proposal — with a learned, state-dependent proposal distribution. This design contrasts with “heavyweight” improvement architectures such as the Dual-Aspect Collaborative Transformer (Ma et al. 2021), which replace the heuristic logic entirely with a deep encoder: our policy remains deliberately small, so that the guided search retains a per-step cost close to that of classical SA and can be batched over thousands of instances on a single GPU.

Simulated Annealing (Kirkpatrick, Gelatt, and Vecchi 1983) iteratively explores the neighborhood of a current solution. Given a solution s , SA samples a neighboring solution $s' \in \mathcal{N}(s)$ and accepts it according to the *Metropolis criterion*: improving moves are always accepted, while worsening moves are accepted with probability $\exp(-\Delta C/T)$, where $\Delta C = C(s') - C(s)$ is the cost difference and T the current temperature. The temperature decays from an initial value T_0 to a final value T_{final} over the search horizon according to a monotone cooling schedule, which regulates the transition from exploration to exploitation.

Neighboring solutions are produced by a *neighborhood operator* $H : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$ that maps a solution s and an action $a \in \mathcal{A}(s)$ to the neighbor $s' = H(s, a)$, so that

$$\mathcal{N}(s) = \{H(s, a) : a \in \mathcal{A}(s)\}.$$

In our setting, an action designates a pair of nodes $a = (i, j)$, and H applies a standard pairwise local search move to them (e.g., relocating node i next to node j). The operator is a

Algorithm 1 Learning-Guided Simulated Annealing

Input: instance, initial solution s , policy π_θ , operator H **Output:** best solution found s^*

```
1: Initialize  $s^* \leftarrow s$  and temperature  $T \leftarrow T_0$ 
2: while termination criterion not met do
3:   Encode  $s$  into per-node feature vectors
4:   Sample action  $a \sim \pi_\theta(\cdot | s)$  (guided proposal)
5:   Generate candidate  $s' \leftarrow H(s, a)$ 
6:    $s \leftarrow s'$  with probability  $\min(1, e^{-\Delta C/T})$  (Metropolis)
7:   Update  $s^*$  if  $C(s) < C(s^*)$ ; cool  $T$ 
8: end while
9: return  $s^*$ 
```

modular component of the framework; the candidate operators and their comparison are presented in the experimental section.

Whereas classical SA draws a uniformly over $\mathcal{A}(s)$, LG-SA samples it from a policy π_θ conditioned on the current solution:

$$a \sim \pi_\theta(\cdot | s) \quad \text{and} \quad s' = H(s, a) \in \mathcal{N}(s).$$

Everything else — the operator, the Metropolis acceptance, the cooling schedule — is kept from classical SA. The overall procedure is summarized in Algorithm 1 and illustrated in Figure 1.

The Guided Search as a Markov Decision Process

To learn the proposal distribution with RL, we frame the SA loop as a Markov Decision Process. At step t , the *state* s_t is the current feasible solution together with the search meta-information (current temperature and remaining budget), encoded as a set of per-node feature vectors described below. The *action* $a_t = (i, j)$ is the node pair handed to the operator H . The *transition* is the composition of the operator and the stochastic Metropolis acceptance: the next state is $H(s_t, a_t)$ if the move is accepted and s_t otherwise, so the environment dynamics include the acceptance randomness of SA.

The *reward* is derived from the evolution of the solution cost along the search: a move that is accepted and improves the solution receives a positive reward proportional to the (normalized) cost reduction, a rejected move yields no improvement signal, and proposals leading to infeasible neighbors can be penalized. Reward variants that instead emphasize improvements of the best solution found so far, or the final solution quality at the end of the horizon, fit the same framework; the precise shaping is a design choice studied experimentally. The agent thus maximizes the expected cumulative discounted reward over the SA horizon, which aligns the learned proposal with the overall goal of driving the annealing process toward low-cost solutions rather than with any single greedy step.

Node-Centric State Representation

To be applicable to any problem size N and to remain *scale-invariant*, the state is encoded in a *node-centric* fashion:

each node i carries its own local feature vector $\mathbf{x}_i$, and no fixed-size global descriptor is used. The features fall into four groups:

- *Static instance features*: node coordinates, polar coordinates relative to the depot, a depot indicator, and the normalized demand q_i/Q ;
- *Solution topology and local geometry*: the coordinates of the node’s current predecessor and successor in its route, together with local quality signals such as the detour cost incurred by the node at its current position, its distance to the route centroid, and local density statistics of the instance;
- *Capacity status*: the load ratio and remaining slack of the node’s current route, and the node’s own contribution to that load;
- *Search meta-features*: the current (normalized) SA temperature and the fraction of the search budget remaining, which let the policy adapt its behavior to the phase of the annealing schedule.

All features are local to a node and standardized, so the same trained model can be deployed on instances of arbitrary size without retraining. The complete list of features, with their formal definitions, is given in the appendix; their individual contribution is analyzed in the experimental section.

Two-Stage Conditional Policy

The policy must define a distribution over the $\mathcal{O}(N^2)$ node pairs while remaining cheap enough to be evaluated at every SA step. We therefore factorize the joint selection as

$$P(i, j | s) = P(i | s) \cdot P(j | i, s),$$

and implement it with a lightweight two-stage architecture, illustrated in Figure 2. In the first stage, a small network f_θ maps each feature vector to a scalar score $z_i = f_\theta(\mathbf{x}_i)$, and the first node is sampled from the softmax distribution over these scores:

$$i \sim P(\cdot | s) = \text{softmax}(\{z_i : i \in \mathcal{C}\}).$$

In the second stage, the representation of the selected node i is combined with that of every other node j , and a second network g_θ assigns a conditional score $z_{ij} = g_\theta(\mathbf{x}_i, \mathbf{x}_j)$ from which the partner is sampled:

$$j \sim P(\cdot | i, s) = \text{softmax}(\{z_{ij} : j \in \mathcal{C}\}).$$

Both stages are built from small multilayer perceptrons applied identically to every node, which makes the policy permutation-equivariant. The second stage can additionally be conditioned on cheap pairwise descriptors of the candidate move (e.g., the exact cost of the move joining i and j), computed only for the selected i .

This factorization also provides a natural hook for the capacity constraints: once i is selected, the partners j for which $H(s, (i, j))$ would be infeasible are masked out of the second-stage distribution, so every proposed neighbor is feasible at the price of $\mathcal{O}(N)$ feasibility checks per step. Alternative constraint-handling strategies are discussed and compared in the experimental section.

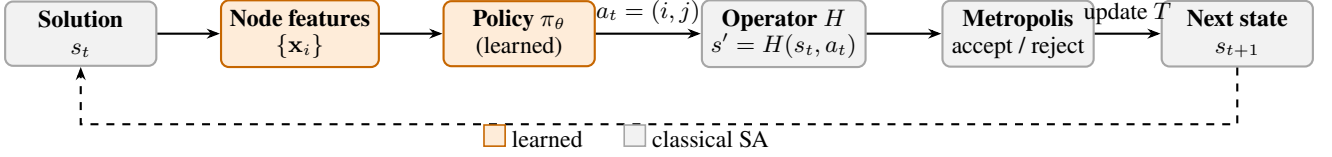


Figure 1: The LG-SA loop. Classical SA components (gray) are kept unchanged: the operator H , the Metropolis acceptance, and the cooling schedule. The only modified block is the move proposal (orange): the current solution is encoded into per-node feature vectors and a learned policy samples the node pair $a_t = (i, j)$, replacing the uniform sampling of classical SA.

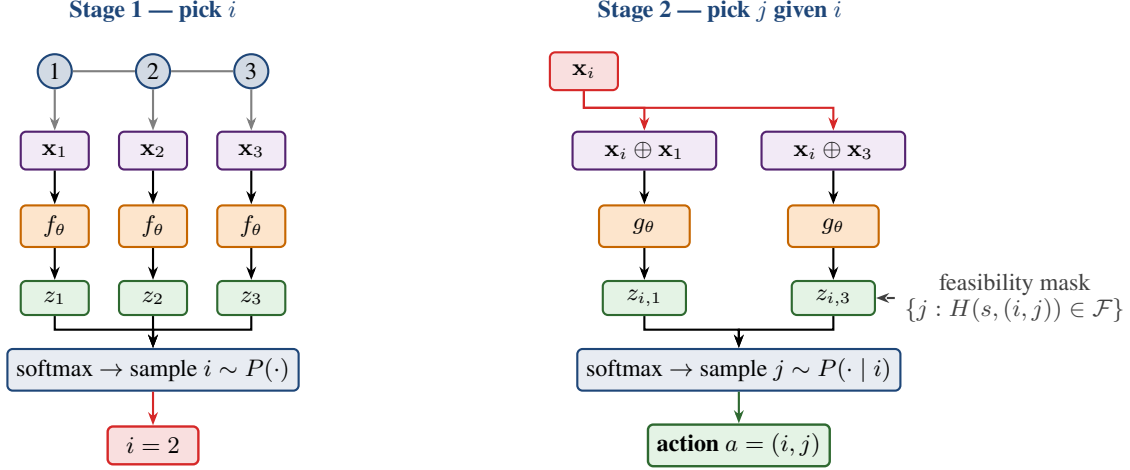


Figure 2: Two-stage conditional sampling of the node pair (i, j) , shown on a toy route of three customers. Stage 1: every node’s feature vector is scored independently by a shared network f_θ and the first node i is sampled from the softmax over the scores. Stage 2: the representation of the selected node i is combined with that of every candidate partner j and scored by a second network g_θ ; infeasible partners are masked out before the softmax, and j is sampled from the conditional distribution. The factorization $P(i, j) = P(i) \cdot P(j | i)$ makes each proposal $\mathcal{O}(N)$.

Since the conditional network is evaluated only for the single selected first node, each proposal costs $\mathcal{O}(N)$ — as opposed to the $\mathcal{O}(N^2)$ cost of scoring all pairs jointly — and the whole step is expressed as batched tensor operations, allowing thousands of instances to be advanced in parallel on one GPU.

Training with PPO

The policy is trained with *Proximal Policy Optimization* (PPO) (Schulman et al. 2017), the standard on-policy actor-critic algorithm. Training alternates between a *collection* phase, in which the current policy runs the full LG-SA loop of Algorithm 1 on a large batch of freshly generated instances in parallel and stores every transition (s_t, a_t, r_t) , and an *update* phase, in which the stored episodes are replayed for several optimization epochs. Advantages are estimated with Generalized Advantage Estimation, and the actor maximizes the usual clipped surrogate objective with an entropy bonus; we refer to Schulman et al. (2017) for the standard formulation and only detail our departures from it.

The critic V_ϕ is a separate permutation-invariant network: it encodes each node feature vector independently, pools the embeddings across nodes into a fixed-size summary, and maps this summary to a scalar value estimate. Keeping the

actor and critic distinct prevents interference between the policy and value representations.

Our implementation departs from standard PPO in three ways, all aimed at stabilizing training on the long and noisy SA horizon:

- *Hybrid clipping–KL objective.* Rather than choosing between the clipped surrogate and the KL-penalized objective, we use both: an adaptive Kullback–Leibler penalty $\beta_{\text{KL}} D_{\text{KL}}(\pi_{\theta_{\text{old}}} || \pi_\theta)$ is added to the clipped loss, and its coefficient is doubled or halved after each update epoch depending on whether the measured divergence leaves a band around a target value. This soft trust region allows many optimization epochs on each collected batch without catastrophic policy drift.
- *KL-coupled learning rate.* The same divergence signal modulates the actor’s learning rate within fixed bounds — gently decreased when the policy moves too fast, increased when it stalls — so the effective step size tracks the trust region automatically.
- *Regularized critic.* The value loss is clipped, mirroring the actor’s trust region, and augmented with a gradient penalty that encourages a smooth (near-Lipschitz) value landscape; we found this useful given that consecutive

states differ by a single local move whose acceptance is itself stochastic.

Experimental Results

Experimental Setup

Benchmark. All experiments reported in this section use the uniform benchmark distribution of Nazari et al. (2018): depot and customer coordinates drawn uniformly in $[0, 1]^2$, demands drawn uniformly in $\{1, \dots, 9\}$, and a size-dependent vehicle capacity ($Q = 20, 30, 40, 50$ for $N = 10, 20, 50, 100$). For each size we pre-generate a fixed test set of 10,000 instances that is reused by every configuration, so that all comparisons are paired at the instance level. Unless stated otherwise, results are reported at $N = 100$.

Evaluation protocol. A trained model is evaluated by running the LG-SA loop over the full test set in parallel on a single GPU, starting from random feasible solutions and running 10,000 SA steps per instance with half-precision inference. We report the mean final routing cost over the 10,000 instances and the wall-clock time of the whole batch. Classical (blind) SA — uniform proposals with the same operator, step budget, and cooling schedule — is evaluated under the identical protocol, as is an OR-Tools baseline with a fixed per-instance time limit (Furnon and Perron 2024).

Statistical protocol. Every configuration is trained under 3 random seeds (5 for confirmation and headline runs), and we report means over seeds. The seed-to-seed spread of the reference configuration, measured once by repeated training and evaluation of the baseline, serves as the empirical noise floor: differences below this threshold are treated as ties.

Sequential validation protocol. The design space of LG-SA — operator, feasibility strategy, features, architecture, reward, annealing parameters, initialization, optimization budget, curriculum — is far too large for a full factorial study. We therefore validate it by coordinate descent: the axes are swept one at a time in order of expected leverage, and the winner of each sweep is frozen into the configuration before the next axis is examined, so later sweeps inherit all earlier decisions. Targeted follow-up batches probe the interactions we consider most at risk (e.g., the construction of the minimal feature set and the curriculum mechanism ablation), and the complete winning configuration is re-trained from scratch with 5 seeds as a final confirmation.

Local Search Operators and Feasibility Strategies

The first axis fixes the foundational search mechanics: the operator H and the way capacity constraints are enforced.

Operators. We consider three standard pairwise operators, illustrated in Figure 3:

- *Swap*: exchanges the positions of nodes i and j , within a route or across routes;
- *Insertion (relocation)*: removes node i from its current position and reinserts it immediately after node j ;
- *2-opt / 2-opt**: for intra-route pairs, reverses the path segment between i and j ; for inter-route pairs, exchanges the route suffixes following i and j (tail swap).

The operator is fixed for a given model: the policy is trained to propose moves for one operator, and learns a distribution tailored to it.

Feasibility strategies. Under capacity constraints, many actions produce an infeasible neighbor $H(s, a) \notin \mathcal{F}$ — the central obstacle in extending neural SA from unconstrained problems to the CVRP. We compare three ways of keeping the search inside $\mathcal{F}$:

- *Proactive filtering (action masking)*. The policy is restricted to the feasible subset $\mathcal{A}_f(s) = \{a : H(s, a) \in \mathcal{F}\}$ by masking infeasible actions before sampling. The conditional factorization makes this cheap: the first node i is selected freely, and only then are the feasible partners $\{j : H(s, (i, j)) \in \mathcal{F}\}$ computed and used to mask the second stage — $\mathcal{O}(N)$ checks per step instead of $\mathcal{O}(N^2)$. Every proposal is feasible by construction; since the fleet is unlimited, a node can always open a new route, so the feasible set is never empty.
- *Reactive rejection*. The policy samples any action from its learned distribution and the resulting neighbor is checked afterwards: if $s' \notin \mathcal{F}$, the move is rejected and the current solution retained. This avoids the filtering overhead but can waste both compute and learning signal when many sampled moves are infeasible.
- *Indirect representation*. Feasibility is delegated to the representation: the solution is encoded as an unconstrained permutation of the customers, the policy modifies the permutation, and a deterministic *greedy split* heuristic packs it back into capacity-feasible routes (Vidal 2016). Every state is feasible by construction and the policy never handles constraints, but the reconstruction runs at every step and its cost grows with the problem size.

We train one policy for each of the nine operator–strategy combinations under the common protocol; the winning pairing is carried unchanged through all subsequent experiments.

State Feature Ablation

The node representation is validated with three complementary protocols, all trained on the winning operator–feasibility pairing and all operating on the feature groups defined in the appendix. *Leave-one-out* removes a single group from the full set and measures the degradation; *thematic* ablations remove a whole correlated cluster at once (e.g., all three density scales, or all capacity signals), exposing groups whose members are individually redundant but collectively useful; *isolation* adds a single group to a minimal base containing only the static and meta features, measuring its standalone value. From these measurements we construct candidate minimal feature sets — keeping groups whose removal hurts, dropping those that fail both the leave-one-out and the isolation test, and keeping one representative per correlated cluster — and confirm the candidates seed-paired against the full set. Finally, the conditional pair descriptors of the second stage (edge distance-ranks and exact insertion cost, defined in the appendix) are evaluated on top of the chosen set, individually and combined.

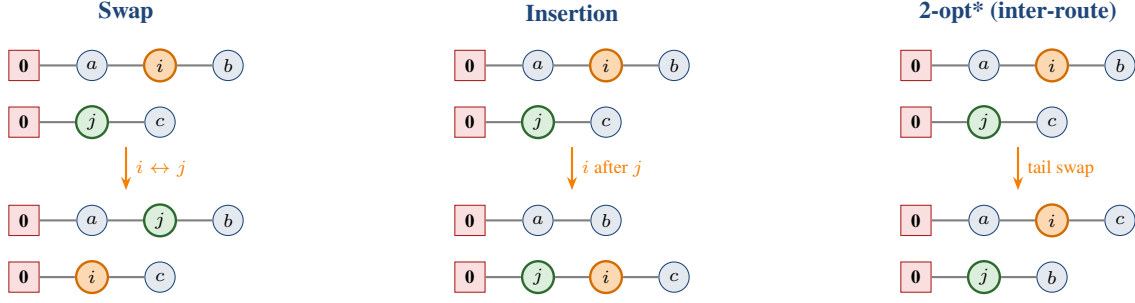


Figure 3: The three pairwise operators on two route fragments (top: before, bottom: after). *Swap* exchanges i (orange) and j (green). *Insertion* removes i and reinserts it immediately after j . *2-opt** exchanges the route suffixes following i and j ; when i and j belong to the same route, the operator instead reverses the path segment between them (standard 2-opt).

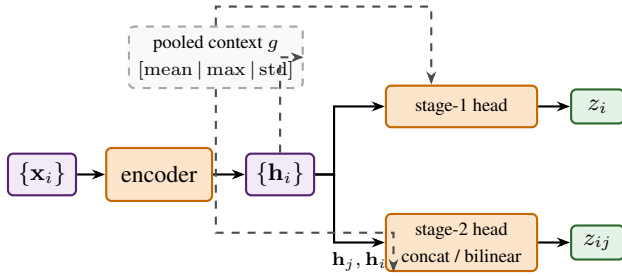


Figure 4: Shared-encoder actor variant. Each node is encoded once into an embedding $\mathbf{h}_i$; two lightweight heads then score the two stages over the shared embeddings, either by concatenation or as a bilinear compatibility between the selected node and each candidate. An optional pooled summary g of all node embeddings (dashed) is appended to both heads’ inputs, giving every score access to instance-level context.

Policy and Critic Architecture

The architecture axis is swept in four steps, each inheriting the previous winner.

Critic. The baseline critic scores each node with a small MLP and averages the scalar outputs into the value estimate, which makes V_ϕ a linear functional of the mean node representation. The alternative is the multi-statistic variant of the training section: nodes are encoded into embeddings, pooled with mean, max, and standard deviation, and mapped to the value by a nonlinear head — so dispersion information (clustered against scattered instances) survives the pooling.

Capacity. The embedding width and the number of hidden layers of the scoring networks are swept jointly, trading representational capacity against per-step inference cost.

Structure. The incumbent actor applies two independent MLPs directly to the raw feature vectors (Figure 2). We compare it with a *shared-encoder* variant (Figure 4): each node is encoded once into an embedding $\mathbf{h}_i$, and both stages are lightweight heads over the shared embeddings, the conditional score being computed either by a concatenation head over $[\mathbf{h}_j \parallel \mathbf{h}_i]$ or as a bilinear compatibility $\mathbf{q}^\top \mathbf{k} / \sqrt{d}$ between

a query built from the selected node and a key per candidate. Orthogonally, a *global context* option pools the node representations into a summary g (mean, max, and standard deviation) appended to the input of both stages — the cheapest way to give the per-node scores access to instance-level information, at one pooling operation per step.

Distribution shaping. Finally, two flags tame the proposal distribution without adding capacity: *logit clipping*, which bounds the logits to $C \tanh(z)$ before the softmax so the distribution cannot collapse to a near-deterministic choice (Kool, van Hoof, and Welling 2019), and a *learnable softmax temperature* per stage. These are judged on policy entropy and update stability in addition to final cost.

Reward, Annealing, and Acceptance

We then validate the remaining components of the search loop.

Reward shaping. Let Δ_t denote the cost improvement realized by the executed transition (zero when the move is rejected) and B_t the best cost found so far. The compared variants are: *immediate*, $r_t = \Delta_t$ with a fixed penalty for infeasible proposals; *acceptance-aligned*, which rewards accepted improving moves by Δ_t but deliberately assigns zero to accepted worsening moves (exploration is SA’s responsibility, not the policy’s fault) and a small constant penalty to rejected proposals; *best-so-far*, $r_t = \max(0, B_{t-1} - B_t)$, which only rewards new records; *terminal*, a single end-of-episode reward equal to the relative improvement over the initial cost; *mixtures*, either a fixed convex combination of the immediate and best-so-far signals or a scheduled interpolation that starts from the dense acceptance-aligned signal early in training and shifts toward the sparser best-so-far or terminal signals; and *step-penalty*, a negative per-step reward proportional to the current best cost, pressuring fast descent.

Annealing. We compare cooling schedule families (geometric decay $T_{k+1} = \alpha T_k$ against step decay) and sweep the temperature range (T_0, T_{final}) , which controls how much worsening SA tolerates at the start and how strict it becomes at the end.

Acceptance. A 2×2 ablation enables or disables the Metropolis criterion independently at training and at test

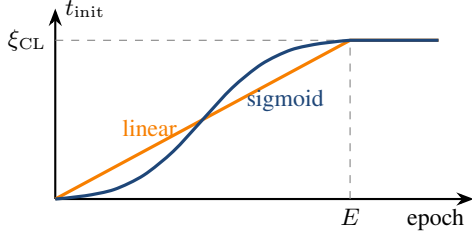


Figure 5: Curriculum ramp: the warm-up depth t_{init} grows from 0 to the warm-up strength ξ_{CL} over E training epochs, following a linear or sigmoid schedule, and stays saturated afterwards.

time (its “off” arm accepts every proposed move), quantifying whether the learned proposal replaces the stochastic acceptance or complements it.

Proposal mode at inference. In standard RL applications, trained policies are often exploited greedily by selecting the most probable action. Our framework deviates from this practice: since the policy acts as the proposal distribution of an SA step, we retain stochastic sampling, $a_t \sim \pi_\theta(\cdot | s_t)$, at inference as well. A greedy policy would repeatedly propose the same “best” move, collapsing the diversity that the Metropolis criterion needs to balance exploration and exploitation; the ablation compares sampling against greedy arg-max exploitation.

Initialization. Each training episode starts from an initial solution produced by a constructive heuristic: random assignment, nearest neighbor, sweep, or single-customer routes. We compare training from each single heuristic against a progressively introduced mixture of all four — mixing exposes the policy to qualitatively different regions of the solution space and prevents it from overfitting to the basin of one construction heuristic. Every trained policy is additionally cross-tested from all initialization types, measuring how much the final quality depends on the starting point.

Optimization Hyperparameters

The optimization knobs — actor and critic learning rates, entropy coefficient, and the rollout length (the number of SA steps collected per PPO update, which sets the horizon over which advantages are estimated) — are swept on the frozen configuration. The number of PPO epochs per collected batch is examined last and re-visited jointly with the curriculum in the final model, since the two interact.

Curriculum Learning

Training episodes are much shorter than the search budgets used at deployment, so a policy trained only from raw constructive solutions rarely visits the fine-grained regime near local optima that dominates the end of a long search. The curriculum closes this gap by moving the *starting point*: before each collection phase, the batch of instances is first warmed up by the current policy for t_{init} steps, and t_{init} grows over training from 0 to a maximum warm-up strength ξ_{CL} over E

epochs, following a linear or sigmoid ramp (Figure 5). Early in training the policy thus learns to repair poor solutions; later it increasingly learns to improve already-refined ones. A further variant draws a per-instance depth $d_i \sim \mathcal{U}\{0, \dots, t_{\text{init}}\}$ instead of warming every instance to the same depth, so that a single batch covers the entire spectrum from raw initializations to deeply improved solutions.

The curriculum is validated with its own sequential protocol: (0) a multi-seed no-curriculum baseline, which also provides the noise floor used throughout this section; (1) a go/no-go comparison of a neutral mid-range curriculum against that baseline; (2) schedule shape (sigmoid against linear); (3) warm-up strength ξ_{CL} ; (4) ramp duration E ; (5) sigmoid steepness; (6) a mechanism ablation opposing the ramped warm-up to a constant warm-up of equal budget, separating the contribution of the progression itself; and (7) the per-instance random depth, hypothesized to pay off in robustness and out-of-distribution behavior more than in the in-distribution headline. The winning curriculum is confirmed against the baseline with 5 seeds.

Final Model and Comparison with Baselines

All component winners, the best curriculum, and the tuned PPO budget are combined into the final model, trained with 5 seeds; the best checkpoint provides the headline numbers. We compare it against blind SA under matched conditions (same operator, budget, and schedule) and against the OR-Tools baseline, and we report the mean cost as a function of the SA step budget from 10^2 to 10^6 steps to characterize the anytime behavior of the guided search.

Generalization and Scalability

Finally, we probe how far the node-centric representation carries across problem scales: models trained at each size are cross-tested at every other size, and the final model is evaluated on instances well beyond its training size with the same checkpoint, reporting both solution quality and runtime scaling.

Conclusion

Appendix: State Feature Definitions

This appendix defines every component of the node feature vector $\mathbf{x}_i$ used by the policy and the critic. Throughout, $\text{prev}(i)$ and $\text{succ}(i)$ denote the nodes immediately before and after i in the current visiting order, $r(i)$ the route currently serving i , $Q_{r(i)}$ the total load of that route, and $d(\cdot, \cdot)$ the Euclidean distance. Figure 6 illustrates the geometric quantities. All features are normalized to comparable ranges, and each is computed locally at node i : the encoding contains no fixed-size global descriptor, which is what makes the representation applicable to any instance size.

Static instance features (6). The 2D coordinates $\mathbf{p}_i = (p_i^x, p_i^y) \in [0, 1]^2$; a depot indicator $\mathbf{1}_{\{i=0\}}$; the polar angle θ_i of the node around the depot; the distance to the depot d_{i0} , min-max normalized per instance; and the demand normalized by the vehicle capacity, q_i/Q . These features depend only on the instance, not on the current solution.

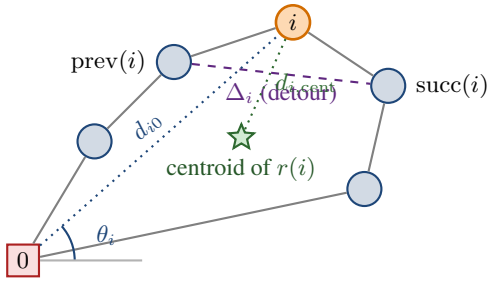


Figure 6: Geometric features attached to a node i (orange) within its current route. The detour Δ_i measures the extra length caused by visiting i between its neighbors; (θ_i, d_{i0}) locate i in polar coordinates around the depot; $d_{i,\text{cent}}$ measures how far i lies from the center of gravity of its own route.

Route topology (4). The coordinates $\mathbf{p}_{\text{prev}(i)}$ and $\mathbf{p}_{\text{succ}(i)}$ of the node’s current neighbors in the visiting order. They give the policy direct access to the local shape of the route around each node.

Neighbor distance-ranks (2). For each of $\text{prev}(i)$ and $\text{succ}(i)$, the rank of that neighbor in the list of all nodes sorted by distance from i , normalized to $[0, 1]$ (the nearest node has rank ≈ 0). A well-placed node is linked to low-rank neighbors; a high rank signals a locally poor connection, independently of the absolute scale of the instance.

Relative demand (1). The demand normalized by the largest demand in the instance, $q_i / \max_{j \in C} q_j$, complementing q_i / Q with a signal that is invariant to the capacity level.

Spatial density (3). The mean distance from i to its k nearest neighbors, evaluated at three scales: $k = 5$, $k = \lceil N/10 \rceil$, and $k = \lceil N/3 \rceil$. These static statistics distinguish nodes in dense clusters from isolated ones.

Local solution quality (2). The *detour cost*

$$\Delta_i = d(\text{prev}(i), i) + d(i, \text{succ}(i)) - d(\text{prev}(i), \text{succ}(i)),$$

i.e., the extra length the tour pays for visiting i at its current position (equivalently, the saving obtained by removing it); and the distance $d_{i,\text{cent}}$ from i to the centroid of the nodes of its route, which flags spatial outliers assigned to the wrong route.

Capacity status (3). The load ratio of the node’s route, $L_{r(i)} = Q_{r(i)} / Q$; the remaining slack, $S_{r(i)} = (Q - Q_{r(i)}) / Q$; and the node’s share of its route load, $\eta_i = q_i / Q_{r(i)}$. These dynamic signals indicate whether a route is nearly saturated and how much a given node contributes to that saturation.

Search meta-features (2). The current SA temperature, normalized to $[0, 1]$ over the schedule range, and the fraction of the search budget remaining. They are identical across nodes of the same state and let the policy modulate its behavior along the annealing schedule (e.g., bolder restructuring at high temperature, fine repairs near the end).

Group	Features	Dim
Static	$\mathbf{p}_i, \mathbf{1}_{\{i=0\}}, \theta_i, d_{i0}, q_i / Q$	6
Topology	$\mathbf{p}_{\text{prev}(i)}, \mathbf{p}_{\text{succ}(i)}$	4
Neighbor ranks	rank of $\text{prev}(i), \text{succ}(i)$	2
Relative demand	$q_i / \max_j q_j$	1
Spatial density	$\mu_5, \mu_{N/10}, \mu_{N/3}$	3
Local quality	$\Delta_i, d_{i,\text{cent}}$	2
Capacity status	$L_{r(i)}, S_{r(i)}, \eta_i$	3
Meta	temperature, remaining budget	2
Total		23

Table 1: Summary of the node feature vector $\mathbf{x}_i$.

Conditional pair features (stage 2, optional). In addition to the per-node vectors above, the second stage of the policy can receive descriptors of the candidate move itself, computed only for the selected first node u and every candidate partner v : the exact insertion cost $d(v, u) + d(u, \text{succ}(v)) - d(v, \text{succ}(v))$ together with the net cost change of the full move (insertion cost minus the removal saving of u), and the normalized distance-ranks of the edges the move would create. Because they are evaluated for a single u , these descriptors preserve the $\mathcal{O}(N)$ per-step complexity.

References

- Correia, A. H. C.; Worrall, D. E.; and Bondesan, R. 2023. Neural Simulated Annealing. In *Proceedings of The 26th International Conference on Artificial Intelligence and Statistics*, 4946–4962. PMLR. ISSN: 2640-3498.
- Furnon, V.; and Perron, L. 2024. OR-Tools Routing Library.
- Kirkpatrick, S.; Gelatt, C. D.; and Vecchi, M. P. 1983. Optimization by Simulated Annealing. 220.
- Kool, W.; van Hoof, H.; and Welling, M. 2019. Attention, Learn to Solve Routing Problems! In *International Conference on Learning Representations*.
- Ma, Y.; Li, J.; Cao, Z.; Song, W.; Zhang, L.; Chen, Z.; and Tang, J. 2021. Learning to Iteratively Solve Routing Problems with Dual-Aspect Collaborative Transformer. In *Advances in Neural Information Processing Systems*, volume 34, 11096–11107. Curran Associates, Inc.
- Nazari, M.; Oroojlooy, A.; Snyder, L.; and Takac, M. 2018. Reinforcement Learning for Solving the Vehicle Routing Problem. In *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc.
- Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; and Klimov, O. 2017. Proximal Policy Optimization Algorithms. ArXiv:1707.06347 [cs].
- Vidal, T. 2016. Technical note: Split algorithm in $\mathcal{O}(n)$ for the capacitated vehicle routing problem. *Computers & Operations Research*, 69: 40–47.